

Machine Learning of Quantum Dynamics

David Florin Zoicas

Bachelor's Thesis

Machine Learning of Quantum Dynamics

Bachelor's Thesis

Supervised by Prof. Dr. Frank Pollmann
Professorship of Theoretical Quantum Physics
Department of Physics
Technical University of Munich

Author David Florin Zoicas
Freisinger Landstraße 45a
85748 Garching b. München

Submitted on November 9, 2025

Abstract

Simulating quantum systems classically, in general, requires many resources and is time-consuming. A trusted and reliable method to extrapolate the behavior of a quantum system for longer periods of time based on its past dynamics is linear prediction. That being said, much progress has been made in machine learning over the past few years, especially in the domain of neural networks for sequence modeling. In this thesis, we explore the usage of Temporal Convolutional Networks(TCN) in the context of extrapolating the two-point correlator of an Ising chain. To judge the quality of our predictions, we numerically compute the true dynamics of the Ising chain up to $T_{\max,2}$, after which we compute the dynamics up to $T_{\max,1}$ with $T_{\max,1} \leq T_{\max,2}$, and then we apply linear prediction and our TCN to create the missing values for our time series. Finally, we measure the error of the two predictions to see which method is more reliable.

Contents

1	Introduction	1
2	Theory of the quantum Ising model	3
2.1	Exact solution of the TFIC	3
2.2	Integrability	8
2.3	Perturbation spread: two-point-correlators and Lieb-Robinson bounds . . .	10
3	MPS methods	13
3.1	Entanglement Entropy and Area law	13
3.2	Matrix Product State (MPS)	15
3.3	Time Evolving Block Decimation (TEBD)	17
3.4	Linear Prediction	19
3.5	Temporal Convolutional Networks (TCN)	21
4	Benchmarks	23
4.1	Overall pipeline	23
4.2	Datasets and splits	24
4.3	Model structure of our TCN	26
4.4	Baselines	26
4.5	Extrapolation tasks	27
5	Results	28
5.1	Generalizing across various Ising chains	28
5.2	Effects of Integrability and lack thereof	30
5.3	Generalizing across different phases	32
5.4	Generalizing within the same phase	33
5.5	Probing the limits of TCN	34
5.6	Conclusion	36

1

Introduction

Strongly correlated quantum many-body systems display rich physics and dynamics that remain challenging to predict and interpret. Unlocking the physics behind these systems is central to condensed matter physics and simulating quantum systems on classical computers. These laws manifest in many experimental platforms, including ultracold atoms in optical lattices [1], [2], trapped ions [3], and neutron scattering on quantum magnets [4]. However, as of now, only a limited number of systems have been solved exactly.

One very useful example is the transverse-field Ising chain. Without a longitudinal field, this model is exactly solvable via the Jordan-Wigner transformation which maps spins to non-interacting fermions and applying the Bogoliubov Transformation to diagonalize the Hamiltonian [5]. The integrability of this system makes it an important benchmark for both theory and experiment. That being said, as soon as one adds a longitudinal field, the system becomes non-integrable, i.e. the number of conserved quantities is reduced to a few global symmetries, and ceases to be diagonalizable.

This change in integrability manifests itself through its significant impact on the system dynamics: If the model is integrable, there are many conserved quantities that restrict the system into a small part of the Hilbert space; more specifically only states with the same values for all those conserved quantities as the initial state can actually show up. That's why integrable systems relax to a generalized Gibbs ensemble (GGE) [6], which respects said constraints. In contrast, when the model is non-integrable, one loses most of those conserved quantities, so the system has access to a much bigger part of Hilbert space and ends up relaxing to the classical Gibbs ensemble. The difference is huge: integrable systems tend to have neat, structured dynamics with stable quasi-particles, whereas non-integrable ones exhibit more chaotic dynamics and look much more thermal.

To analyze these out-of-equilibrium dynamics, numerous numerical algorithms have been de-

veloped, such as Time Evolution Block Decimation (TEBD) or Density-Matrix Renormalization Group (DMRG) [7]. These methods operate on tensor network representations of quantum states and can track their evolution in time. However, as time progresses, the entanglement entropy grows, and the computational resources required scale exponentially with it, severely limiting reachable simulation times. This forces us to set a maximal bond order, to limit the maximal entanglement entropy. That only works for a limited amount of time steps, until the truncation error of the two-point correlator grows too large and the simulation ceases to describe any meaningful physics.

To circumvent this computational limitation, one can rely on algorithms such as Linear prediction [8, 9] to extrapolate e.g. two point correlators beyond the maximal time to which they have been computed, and predict with good accuracy how the system evolves for large times, allowing one to e.g. compute sharper spectral functions [9]. However, such techniques are quite antiquated, and much progress has been made in the domain of Machine Learning and Deep Learning.

Many models have been developed with the intention of tackling sequential problems, such as extrapolating time series. We will focus on a particular class of convolutional networks called Temporal Convolutional Networks and explore their ability to help us understand the rich dynamics of one dimensional quantum systems such as Ising Chains with both transversal and longitudinal fields. We will be particularly exploring the role of integrability in the ease of the neural network to understand the complex physics that drive the time evolution of our two-point correlator. We have selected Temporal Convolutional Networks due to their superior performance on sequential tasks relative to sequential networks such as RNN's and LSTM's, which has been explored and discussed by Bai et. al. in [10].

In Section 2, we discuss the transverse-field Ising chain's exact solution derived by Pfeuty [5], discuss its implications, and analyze its spectral function. We will then discuss the implications of integrability or the lack thereof on the transverse-field Ising Chain's dynamics. In Section 3 we dive into how quantum states of a transverse-field Ising chain can be represented as Matrix-Product states (MPS) and the numerical approaches to computing the ground state of the Ising Chain as well as the two-point correlator as a function of time. We discuss the mathematics behind Linear Prediction as well as why this ansatz is a valid approach for extrapolating this particular timeseries. In Section 4 we will shortly address the history of neural networks in performing extrapolation of time series. Then we will dive into the mathematical structure of TCNs and discuss the parameters as well as the training strategies used for creating our model. Finally in Section 5 we present our results as well as an interpretation on the physics of our various systems based on the systems ability to grasp their respective patterns.

2

Theory of the quantum Ising model

In general, a quantum Ising chain is described by

$$H = -J \sum_{i=1}^L S_i^z S_{i+1}^z - h_x \sum_{i=1}^L S_i^x - h_z \sum_{i=1}^L S_i^z, \quad (2.1)$$

where $J > 0$ is the interaction constant of the z-spins of two $\frac{1}{2}$ -spin particles that are situated on a one-dimensional equidistant lattice, and the constants h_x and h_z represent the transverse and longitudinal magnetic fields applied to our system. Later in our simulations, we will use the spin operators $S^\alpha = \hbar \frac{\sigma^\alpha}{2}$, $\alpha \in \{x, y, z\}$ and $\hbar \equiv 1$. For simplicity's sake, we will switch to Pauli matrices, as this only changes the parameters $J_\sigma = \frac{J_S}{4}$, $h_{x,\sigma} = \frac{h_{x,S}}{2}$ and $h_{z,\sigma} = \frac{h_{z,S}}{2}$. We will only return to using Spin operators after diagonalizing the TFIC in order to get the right spectral function, Lieb-Robinson velocity and critical point ($h_z = 0$) for our simulations. Unless explicitly stated, we set $J \equiv J_\sigma$ and $h \equiv h_\sigma$. Typically, when dealing with a quantum Ising chain, most discussions revolve around the integrable, exactly solvable transverse-field Ising chain with only a transversal field. Whenever one adds a longitudinal field, however, this breaks down and the system starts exhibiting richer, more complex physics. We will first go over the TFIC, then we will discuss the implications of integrability or the lack thereof.

2.1 EXACT SOLUTION OF THE TFIC

The system in question is described by the following Hamiltonian:

$$H = -J \sum_{i=1}^L \sigma_i^z \sigma_{i+1}^z - h_x \sum_{i=1}^L \sigma_i^x, \quad (2.2)$$

This system was solved exactly by Pfeuty [5], by transforming the set of Pauli operators to a set of Fermi operators, and for the following solution we inspired ourselves from [11]. Here,

one has the choice between periodic or open boundary conditions. The exact solution is most conveniently derived for the former with $\sigma_{L+1} \equiv \sigma_1$, which is we will discuss. In our simulations we will use open boundary conditions, however, for a sufficiently large number of lattice sites the two systems behave nearly identical, so this fact becomes negligible. At $T = 0$, H possesses, as we will soon see, two phases which are dependent on h_x .

The exact solution relies on the Jordan-Wigner transformation. This mapping allows us to map our Ising chain with $\frac{1}{2}$ -spin degrees of freedom to spinless fermions. The main takeaway here is that systems with one $\frac{1}{2}$ -spin degree of freedom per site is equivalent to spinless fermions between lattice points. In this case, we may equate the spin-up state of a site with an empty orbital, and the spin-down state with an occupied site, where operators $c_i^\dagger$ and c_i add and remove a spinless fermion to site i respectively. Then, in this new representation, σ_i^z simply becomes:

$$\sigma_i^z = 2c_i^\dagger c_i - 1. \quad (2.3)$$

Here, adding a fermion to site i is equivalent to flipping a spin from down to up with $\sigma_i^+ = \frac{\sigma_i^x + i\sigma_i^y}{2}$, and analogously removing a fermion from site i is equivalent to flipping a spin from up to down, or $\sigma_i^- = \frac{\sigma_i^x - i\sigma_i^y}{2}$.

While this equivalence is valid for single sites, we cannot directly apply it to map our Ising chain to a system of spinless fermions, since two fermionic operators anticommute, but two spin operators commute. Jordan and Wigner solved this problem by introducing the following identities:

$$\sigma_i^+ = \left(\prod_{j=1}^{i-1} 1 - 2c_j^\dagger c_j \right) c_i^\dagger, \quad (2.4)$$

$$\sigma_i^- = \left(\prod_{j=1}^{i-1} 1 - 2c_j^\dagger c_j \right) c_i. \quad (2.5)$$

This representation adds a plus or minus sign in front of the fermionic creation and annihilation operators depending on whether the number of lattice sites to the left of the i -th site that are occupied are even or odd respectively. Note that by merely applying $\left(\prod_{j=1}^{i-1} 1 - 2c_j^\dagger c_j \right)$ to both equations (2.4) and (2.5), we obtain an expression for c_i and $c_i^\dagger$. We can verify that our new fermionic operators satisfy the commutation relationships:

$$\{c_i, c_j^\dagger\} = \delta_{ij}, \quad \{c_i, c_j\} = 0, \quad \{c_i^\dagger, c_j^\dagger\} = 0. \quad (2.6)$$

$$[\hat{\sigma}_i^+, \hat{\sigma}_j^-] = \delta_{ij} \hat{\sigma}_i^z, \quad [\hat{\sigma}_i^z, \hat{\sigma}_j^\pm] = \pm 2 \delta_{ij} \hat{\sigma}_i^\pm, \quad (2.7)$$

This allows us to conveniently write σ_i^x as

$$\sigma_i^x = \left(\prod_{j=1}^{i-1} 1 - 2c_j^\dagger c_j \right) (c_i + c_i^\dagger). \quad (2.8)$$

It is important to note that equations (2.4) and (2.5), alongside the equations

$$\sigma_i^+ \sigma_i^z = -\sigma_i^+, \quad (2.9)$$

$$\sigma_i^- \sigma_i^z = \sigma_i^-, \quad (2.10)$$

allow us to obtain the following useful expressions

$$c_i^\dagger c_{i+1}^\dagger = \sigma_i^+ \sigma_{i+1}^+ \quad (2.11)$$

$$c_i^\dagger c_{i+1} = \sigma_i^+ \sigma_{i+1}^- \quad (2.12)$$

$$c_i c_{i+1}^\dagger = -\sigma_i^- \sigma_{i+1}^+ \quad (2.13)$$

$$c_i c_{i+1} = -\sigma_i^- \sigma_{i+1}^-. \quad (2.14)$$

These expressions will prove useful later when writing the Hamiltonian as a function of Fermi operators. For simplicity's sake, we will perform the following 90° rotation about the y-axis

$$\sigma^z \rightarrow -\sigma^x, \sigma^x \rightarrow \sigma^z, \quad (2.15)$$

which delivers us the modified, more practical Hamiltonian:

$$H = -J \sum_{i=1}^L \sigma_i^x \sigma_{i+1}^x - h \sum_{i=1}^L \sigma_i^z. \quad (2.16)$$

Inserting equations (2.9) and (2.10) into our Hamiltonian gives us the following relation:

$$H = -J \sum_i \left(c_i^\dagger c_{i+1} + c_i^\dagger c_{i+1}^\dagger - c_i c_{i+1} - c_i c_{i+1}^\dagger + 2g c_i^\dagger c_i - g \right), \quad (2.17)$$

where g is defined as $\frac{h_x}{J}$ and H is quadratic in the Fermi operators. This form, however, doesn't tell us much about the systems eigenstates and its conserved Quantities, so we should aim to diagonalize it. An interesting observation is that - while $\sum_i c_i^\dagger c_i$ is not a conserved quantity - the fact that our Hamiltonian is quadratic in Fermi operators implies that Parity is conserved.

We then perform the following Fourier transformation of c_i as such

$$c_k = \frac{1}{\sqrt{L}} \sum_{j=1}^L e^{-ikr_j} c_j. \quad (2.18)$$

to obtain the Hamiltonian in momentum space

$$H = J \sum_k \left(2[g - \cos(ka)]c_k^\dagger c_k + i \sin(ka)[c_{-k}^\dagger c_k^\dagger + c_{-k} c_k] - g \right) \quad (2.19)$$

We now rewrite the Hamiltonian by summing only over $k > 0$, and utilize the vector $\Psi_k^\dagger = \begin{pmatrix} c_k^\dagger & c_{-k} \end{pmatrix}$

$$H = -2J \left(\sum_{k>0} (g - \cos(ka)) \begin{pmatrix} c_k^\dagger c_k - c_{-k} c_{-k}^\dagger \end{pmatrix} + i \cdot \sin(ka) \begin{pmatrix} c_{-k} c_k - c_k^\dagger c_{-k}^\dagger \end{pmatrix} \right) \quad (2.20)$$

$$= -2J \sum_{k>0} \begin{pmatrix} c_k^\dagger & c_{-k} \end{pmatrix} \begin{pmatrix} g - \cos(ka) & i \sin(ka) \\ -i \sin(ka) & -(g - \cos(ka)) \end{pmatrix} \begin{pmatrix} c_k \\ c_{-k}^\dagger \end{pmatrix} \quad (2.21)$$

$$= \sum_{k>0} \mathcal{H}_k, \quad (2.22)$$

which simplifies the Hamiltonian and highlights the role of quasiparticles in the system. Now we aim to map these Fermi operators to Bogoliubov operators η_k that are a superposition of Fermi operators with which we will be able to write our Hamiltonian as

$$H = \sum_k \epsilon_k \eta_k^\dagger \eta_k + \text{const.} \quad (2.23)$$

To this end we define our Bogoliubov operators as

$$\begin{pmatrix} \eta_k \\ \eta_{-k}^\dagger \end{pmatrix} = \begin{pmatrix} u_k & i v_k \\ i v_k & u_k \end{pmatrix} \begin{pmatrix} c_k \\ c_{-k}^\dagger \end{pmatrix} = \begin{pmatrix} u_k c_k + i v_k c_{-k}^\dagger \\ u_k c_{-k}^\dagger + i v_k c_k \end{pmatrix} \quad (2.24)$$

Our coefficients u_k and v_k are determined by two conditions:

1. η_k are fermionic operators
2. The Hamiltonian in equation (2.21) and (2.22) is diagonalized

Because of the anti-commutation relations implied by the first condition, we discover our coefficients must obey

$$u_k^2 + v_k^2 = 1$$

for all k . Inverting equation (2.24) gives

$$\begin{pmatrix} c_k \\ c_{-k}^\dagger \end{pmatrix} = \begin{pmatrix} u_k & -i v_k \\ -i v_k & u_k \end{pmatrix} \begin{pmatrix} \eta_k \\ \eta_{-k}^\dagger \end{pmatrix}, \quad (2.25)$$

which allows us to rewrite our Hamiltonian $\mathcal{H}_k$ as follows:

$$\mathcal{H}_k = \begin{pmatrix} \eta_k^\dagger & \eta_{-k} \end{pmatrix} \begin{pmatrix} u_k & iv_k \\ iv_k & u_k \end{pmatrix} (-2J) \begin{pmatrix} g - \cos(ka) & i \sin(ka) \\ -i \sin(ka) & -(g - \cos(ka)) \end{pmatrix} \begin{pmatrix} u_k & -iv_k \\ -iv_k & u_k \end{pmatrix} \begin{pmatrix} \eta_k \\ \eta_{-k}^\dagger \end{pmatrix} \quad (2.26)$$

Next, we demand the vector $\begin{pmatrix} u_k & -iv_k \end{pmatrix}^\top$ to be an eigenvector of the central matrix in equation (2.26). Solving this problem gives us the eigenvalues

$$\epsilon_k = \mp 2J \sqrt{1 + g^2 - 2g \cos(ka)} \quad (2.27)$$

Inserting equations (2.26) and (2.27) back into equation (2.22) finally gives us

$$H = \sum_{k>0} \epsilon_k (\eta_k^\dagger \eta_k - \eta_{-k} \eta_{-k}^\dagger) \quad (2.28)$$

$$= \sum_k \epsilon_k \eta_k^\dagger \eta_k + E_0. \quad (2.29)$$

We also note that if we used spin operators, the eigenvalues become

$$\epsilon_k = \mp \frac{J_S}{2} \sqrt{1 + 4 \frac{h_{x,S}^2}{J_S^2} - 4 \frac{h_{x,S}}{J_S} \cos(ka)}, \quad (2.30)$$

which will become important later. This is demonstrated in figure 1. It is interesting to note

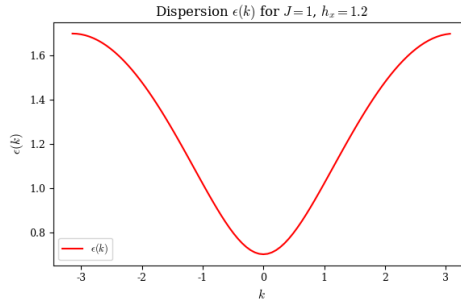


Figure 1: Spectral function of the Ising Chain with parameters $J = 1$ and $h_x = 1.2$.

that this system possesses a second-order phase transition, with the two phases being the ferromagnetic phase and the paramagnetic phase depending on the values of the field h_x . The phase transition is characterized by the energy gap $2J|1 - g|$, where $g = \frac{h_x}{J}$ and the critical point is reached at $g = 1$. In the ferromagnetic phase, the spins align by coupling to their nearest neighbor, and the system's global $\mathbb{Z}_2$ symmetry is broken by a two-fold degenerate ground state, for instance $|\uparrow\rangle \otimes \cdots \otimes |\uparrow\rangle$ and $|\downarrow\rangle \otimes \cdots \otimes |\downarrow\rangle$ in the case of $h_x = 0$. On the other hand, if $g > 1$, the $\mathbb{Z}_2$ symmetry is restored and the transverse field dominates. This causes the spins to couple to the external magnetic field and align along the positive x-axis. For $h \rightarrow \infty$ the ground state converges towards $|\rightarrow\rangle \otimes \cdots \otimes |\rightarrow\rangle$. Also note that when using spin operators, the critical point becomes $\frac{h_{x,S}}{J_S} = \frac{1}{2}$.

This distinction is of great interest to us due to the different types of quasi-particles that occur in these phases, specifically speaking domain walls in contrast to spin flips. This difference may affect the dynamics of transverse-field Ising chain, and therefore could affect the neural networks ability to learn the patterns, so this is one of the objectives we aim to explore.

2.2 INTEGRABILITY

Now in general, a quantum Ising chain possesses the form

$$H = -J \sum_{i=1}^L \sigma_i^z \sigma_{i+1}^z - h_x \sum_{i=1}^L \sigma_i^x - h_z \sum_{i=1}^L \sigma_i^z, \quad (2.31)$$

where $h_z \neq 0$, and this fundamentally changes the dynamics of our system. To see this, we must first understand what Integrable systems are. A system H is integrable when it possesses many integrals of motion I_m whose number typically grows with the size of the system, where our I_m satisfy

$$[H, I_m] = 0, \quad [I_m, I_{m'}] = 0, \quad \langle I_m \rangle = \text{const.}$$

for all m, m' . These conditions imply

1. the eigenstates of our Hamiltonian are eigenstates of our integrals and
2. Our system is restricted to a subset of states that conserve our integrals.

To better understand why this is the case, it is wise to first look at an isolated non-integrable system. In this case, the only integral of motion is the energy, which is represented by the Hamiltonian H itself. We know that such a system will thermalize over time and converge to the Gibbs state

$$\rho_G = Z^{-1} \exp(-\beta H) \quad (2.32)$$

with $Z^{-1} = \text{tr}[\exp(-\beta H)]$ being the partition function. For a detailed discussion, see the review [12].

But now let us suppose we have a set $\{I_m\}$ of integrals as defined above, then we define our wave function to be

$$|\psi(t)\rangle = \sum_{\alpha} C_{\alpha} \cdot e^{-iE_{\alpha}t} |\alpha\rangle \quad (2.33)$$

and impose the starting condition

$$\langle \psi(t) | \mathbf{I} | \psi(t) \rangle = \mathbf{q}$$

where $\mathbf{I} = (I_1 \ I_2 \ \dots)^\top$ and $\mathbf{q} = (q_1 \ q_2 \ \dots)^\top$ are the integrals and the expectation values that remain constant. For any I_m - where for simplicity's sake we set $I_m \equiv I$ - we then obtain

$$\langle I(t) \rangle = \sum_{\alpha, \beta} C_\alpha C_\beta^* e^{-i(E_\alpha - E_\beta)t} I_{\beta\alpha}, \quad (2.34)$$

and if we average over time we get for large t

$$\langle \bar{I} \rangle = \sum_{\alpha} |C_\alpha|^2 I_{\alpha\alpha}. \quad (2.35)$$

With these, we have a set of linear equations which limit the dynamics of our system. In the density matrix formalism, we can then conclude the system, if isolated, will relax locally to the Generalized Gibbs Ensemble (GGE) [6, 12], which is defined as

$$\rho_{GGE} = Z_{GGE}^{-1} \cdot e^{-\sum_m \lambda_m I_m}, \quad (2.36)$$

where Z_{GGE} is the partition function of the GGE and λ_m are Lagrange multipliers that are determined by the conditions described by equation (2.35).

This concept is very important to us because we have seen how the transverse-field Ising chain can be diagonalized with the help of Bogoliubov operators. In addition, from the anti-commutation relationships one can show that the momentum-space density operators $n_k = \eta_k^\dagger \eta_k$ are integrals of motion:

$$[H, n_k] = 0, \quad [n_k, n_{k'}] = 0 \quad \forall k, \quad (2.37)$$

which leads to simpler dynamics such as perturbation spread as we will see in the next section. If, however, we were to introduce a longitudinal field, we would no longer be able to diagonalize the Hamiltonian because terms linear in fermi operators would appear in equation (2.17). As a result, this Ising chain has only few conserved quantities such as energy, crystal momentum and so on. This means that if it were to be perturbed, it would thermalize to the Gibbs state over long periods of time and would exhibit more chaotic behavior in the short term. This is of interest to us because we would like to explore the possibilities neural networks offer us in learning to predict complex quantum behaviors and how integrability affects the networks ability to grasp patterns.

2.3 PERTURBATION SPREAD: TWO-POINT-CORRELATORS AND LIEB-ROBINSON BOUNDS

Now take the transverse-field Ising chain with N sites where $h \gg J$:

$$H = -J \sum_{j=1}^N \sigma_j^z \sigma_{j+1}^z - h_x \sum_{j=1}^N \sigma_j^x \quad (2.38)$$

We also impose open boundary conditions. Now this is the canonical example for the paramagnetic phase, where the transverse field dominates and aligns the spins along the $+x$ axis. The ground state then is

$$|\psi_0\rangle = |\rightarrow\rangle \otimes |\rightarrow\rangle \otimes \dots \otimes |\rightarrow\rangle \quad (2.39)$$

At this point it is wise to observe that a very nice property of ground states is that they have a low entanglement entropy. This is because the entanglement entropy of ground states follow an area law, although what that entails will be explored later in the next section

But suppose we want to introduce a local quench in this Ising chain by e.g. applying a σ^y on the middle spin. What that does is it essentially flips the middle spin (up to a phase factor)

$$\sigma_{\frac{N}{2}}^y |\rightarrow\rangle \otimes \dots \otimes |\rightarrow\rangle \otimes \dots |\rightarrow\rangle \propto |\rightarrow\rangle \otimes \dots \otimes |\leftarrow\rangle \otimes \dots |\rightarrow\rangle. \quad (2.40)$$

This state is no longer energetically favorable, and therefore no longer stable. Now we are interested in exploring how this excitation spreads in our system. The most standard way to do this is to measure the correlation function at every site i at every time step. This is done by calculating

$$C(j, t) = \langle \psi_0 | \sigma_j^y(t) \sigma_{\frac{N}{2}}^y | \psi_0 \rangle = \langle \psi_0 | e^{iHt} \sigma_j^y e^{-iHt} \sigma_{\frac{N}{2}}^y | \psi_0 \rangle. \quad (2.41)$$

For $h_x = 0$, the Hamiltonian becomes

$$H = -J \sum_j \sigma_j^z \sigma_{j+1}^z, \quad (2.42)$$

where all terms are diagonal in the σ^z - basis. Here, one can verify that the Heisenberg equation is

$$[H, \sigma_j^y] = 2iJ(\sigma_{j-1}^z + \sigma_{j+1}^z)\sigma_j^x. \quad (2.43)$$

For simplicity's sake, we choose to split $H = H_{loc} + H_{rest}$, with $[H_{loc}, H_{rest}] = 0$. then the following expression can be derived:

$$\sigma_j^y(t) = e^{iH_{loc}t} \sigma_j^y e^{-iH_{loc}t} = \cos(2Jt(\sigma_{j-1}^z + \sigma_{j+1}^z)) \sigma_j^y + \sin(2Jt(\sigma_{j-1}^z + \sigma_{j+1}^z)) \sigma_j^x, \quad (2.44)$$

so the dynamics is confined to the three sites $\{j-1, j, j+1\}$. In particular, starting from the ferromagnetic product ground state, $C(j, t)$ is on-site oscillatory and vanishes for $j \neq \frac{N}{2}$: there is no spatial spreading and no Lieb-Robinson light cone.

Returning to our paramagnetic example, if we set $h_x > 0$ then upon plotting the correlation function we will be able to observe the domain walls of the Ising chain spreading away from the local quench. This is shown in figure 2.

At this point in time, the reader may or may not intuit what is meant by a Lieb-Robinson

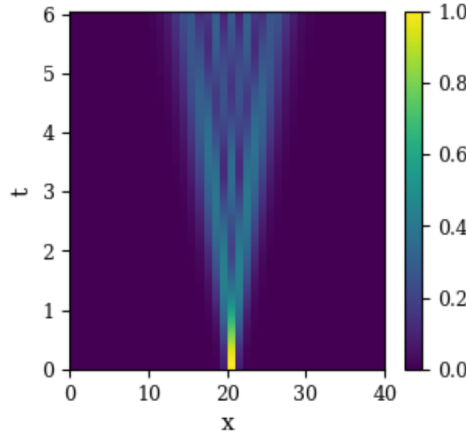


Figure 2: Space-time map of the post-quench correlator $|C(j, t)| = |\langle \psi_0 | \sigma_j^y(t) \sigma_{\frac{N}{2}}^y | \psi_0 \rangle|$ for an open TFIC after a local σ^y flip at the center of the chain.

light cone, but nevertheless, it is something that should be defined mathematically. The Lieb-Robinson theorem states that for local lattice Hamiltonians, there exists a Lieb-Robinson ([13, 14, 15]) (see also the modern treatments and extensions in [16], [17]) velocity v_{LR} such that for local A_x, B_y ,

$$\|[A_x(t), B_y]\| \leq C e^{-\frac{d(x,y)-v_{LR}t}{\xi}},$$

where ξ is the correlation length. By restricting ourselves to the quantum Ising chain, this equation becomes

$$|\langle \sigma_j^y(t) \sigma_{\frac{N}{2}}^y \rangle - \langle \sigma_j^y(t) \rangle \langle \sigma_{\frac{N}{2}}^y \rangle| \leq C e^{-\frac{|j-N/2|-v_{LR}t}{\xi}}, \quad (2.45)$$

where the second term with the $\langle \sigma_{\frac{N}{2}}^y \rangle$ factor is always zero due the Hamiltonian being invariant under the transformation $\sigma^y \rightarrow -\sigma^y$.

If we go back to the Jordan-Wigner formalism, then the speed of the perturbation spread is determined by

$$\epsilon_k = 2J\sqrt{1 + g^2 - 2g \cos(k)}, \quad (2.46)$$

$$v_k = \frac{\partial \epsilon_k}{\partial k} = \frac{2Jg \sin k}{\sqrt{1 + g^2 - 2g \cos k}}. \quad (2.47)$$

We set $a \equiv 1$ for convenience, and one can prove that our spread is limited by

$$v_{\max} = 2J \min(1, g) \quad (2.48)$$

with $v_{\max} \equiv v_{LR}$. In our simulations we will work with spin operators, therefore the maximal speed we are interested in is

$$v_{\max} = \frac{J_S}{2} \min(1, 2 \frac{h_{x,S}}{J_S}). \quad (2.49)$$

As we see in equation (2.50), if the simulation time is chosen to be too large, it will lead to the perturbation reflecting off the edges, thus destroying any possibility to claim our system serves as a good approximation for a transverse-field Ising chain with periodic boundary conditions. Hence, equations (2.47) and (2.48) serves as a guide to appropriately choose the parameters and $T_{\max,2}$ of the simulation, i.e. the condition

$$\frac{N}{2} \gtrsim v_{\max} T_{\max} \quad \Rightarrow \quad N \gtrsim 2 v_{\max} T_{\max}. \quad (2.50)$$

With $v_{\max} = \frac{J_S}{2} \min(1, 2 \frac{h_{x,S}}{J_S})$ this becomes $N \gtrsim J_S \min(1, 2 \frac{h_{x,S}}{J_S}) T_{\max}$, so to be on the safe side we choose

$$N \geq J_S \min(1, 2 \frac{h_{x,S}}{J_S}) T_{\max}. \quad (2.51)$$

As a small aside, what we will attempt to forecast is the correlation function in momentum space, which is simply given by the Fourier transform

$$\tilde{C}(k, t); =; \sum_{j=-(N-1)}^{N-1} e^{-ikr_j} C(r_j, t). \quad (2.52)$$

In fact, the object we ultimately care about is the dynamical spin structure factor $S(k, \omega)$, which is just the time Fourier-transform of our two-point correlator, schematically

$$S(k, \omega) \sim \int dt e^{i\omega t} C(k, t).$$

This quantity is what would be measured in inelastic neutron scattering, so all of our work on forecasting $C(k, t)$ is really about predicting how a real experiment would see the excitation spectrum of the chain, but we will get back to this in section 5.5.

3

MPS methods

Now we wish to have an efficient way to compute the two-point correlator of a quantum Ising chain. Matrix Product States(MPS) [7] alongside time series extrapolation methods, as we will see, enable these calculations numerically.

3.1 ENTANGLEMENT ENTROPY AND AREA LAW

Now let us consider a bipartition of the Hilbert space $\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_B$ which here corresponds to splitting the chain into a block A (sites $1, \dots, l$) and its complement B . For a pure state $|\psi\rangle$, the reduced density matrix of A is

$$\rho_A = \text{Tr}_B(|\psi\rangle\langle\psi|). \quad (3.53)$$

Tr_B is the partial trace over the B subspace of the greater Hilbert space. Then the von Neumann entanglement entropy is defined as

$$S(A) = -\text{Tr}(\rho_A \log \rho_A). \quad (3.54)$$

Using the Schmidt decomposition according to our $A|B$ separation of the Hilbert space

$$|\psi\rangle = \sum_{\alpha} \lambda_{\alpha} |\alpha\rangle_A \otimes |\alpha\rangle_B, \quad \sum_{\alpha} \lambda_{\alpha}^2 = 1, \quad (3.55)$$

one gets

$$S(A) = -\sum_{\alpha} \lambda_{\alpha}^2 \log \lambda_{\alpha}^2 \quad (3.56)$$

due to our density matrix being of the form

$$\rho_A = \sum_{\alpha} \lambda_{\alpha}^2 |\alpha\rangle_A \langle\alpha|_A. \quad (3.57)$$

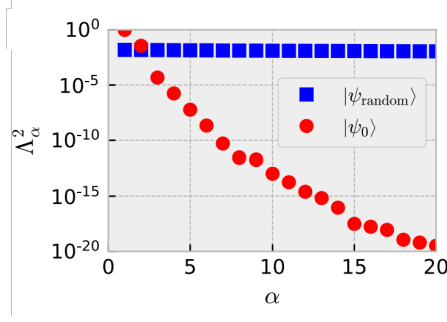


Figure 3: Comparison of the largest Schmidt values of the ground state of the transverse field Ising model ($g = 1.5$) and a random state for a system consisting of $N = 16$ spins. The index α labels different Schmidt values.[7]

Here we can clearly see that S vanishes only if there is only one Schmidt coefficient, and grows the more mixed ρ_A becomes. One note-worthy characteristic of this is if we were to calculate the entanglement entropy $S(B)$, we would still obtain the same result due to the Schmidt coefficients.

As previously discussed, ground states of gapped, local Hamiltonians have low entanglement entropy due to following an area law [18]. This is highly convenient for calculating the dynamics of the system as we will soon see, and concretely this means

$$S(A) \approx \alpha |\partial A| + \dots, \quad (3.58)$$

where α is a constant and ∂A is the set of vertices separating domains A and its complement B , with $|\partial A|$ amounting to being just one in the case of a one-dimensional Ising model. In contrast to such ground states, general states follow a volume law, where the entanglement entropy grows with the dimension of the subspace. In particular, in [19], it has been proven that in a system with N sites and local Hilbert dimension d that is separated in half, a randomly chosen state will have

$$S \approx \frac{N}{2} \log(d) - \frac{1}{2}.$$

At the critical point in 1D, conformal field theory predicts a universal logarithmic scaling of the ground-state entanglement entropy. For open boundaries (OBC) the entanglement entropy between the two halves is

$$S(\ell) = \frac{c}{6} \log \left[\frac{2L}{\pi} \sin \left(\frac{\pi \ell}{L} \right) \right] + s'_1. \quad (3.59)$$

For the Ising universality class, $c = \frac{1}{2}$ [20].

A key observation which is demonstrated in figure 3 is that in states with very low S , a small number of Schmidt states contribute the most to the total state. This can be understood as

there existing some finite χ for any $\varepsilon > 0$ such that

$$\| |\psi\rangle - \sum_{\alpha=1}^{\chi} |\alpha\rangle_A \otimes |\alpha\rangle_B \| < \varepsilon. \quad (3.60)$$

This equation conveniently allows us safely truncate the Schmidt decomposition without losing critical information belonging to the state and lighten the computational burden [21, 7].

3.2 MATRIX PRODUCT STATE (MPS)

Building on our last section, we will discuss matrix product states to see how they lead to an efficient representation of 1D Ising models. MPS are best visualized using Tensor networks, which appear in figure 4 and will be shortly discussed.

Now consider an Ising chain with N sites where $\{j_1, j_2, \dots, j_N\}$ are the quantum numbers describing the state, then a state can in general be written as

$$|\psi\rangle = \sum_{j_1, j_2, \dots, j_N} c_{j_1, j_2, \dots, j_N} |j_1\rangle \otimes |j_2\rangle \otimes \dots \otimes |j_N\rangle, \quad (3.61)$$

with $c_{j_1, j_2, \dots, j_N}$ being the probability amplitude for each eigenstate. Here we can observe that in order to describe this state, we must have 2^N probability amplitudes, which is the limiting factor when it comes to running simulations involving time-evolution of such states.

This is precisely what we previously discussed, and we will explore how we can use MPS to compress our wavefunction to a computationally manageable size. The key of an MPS is that our coefficients $c_{j_1, j_2, \dots, j_N}$ are the results of matrix products of the form

$$|\psi\rangle = \sum_{j_1, j_2, \dots, j_N} M^{[1], j_1} M^{[2], j_2} \dots M^{[N], j_N} |j_1\rangle \otimes |j_2\rangle \otimes \dots \otimes |j_N\rangle. \quad (3.62)$$

The matrices we have chosen are $M^{[n], j_n} \in \mathbb{C}^{\chi_{n-1} \times \chi_n}$. Evidently, we must have $\chi_1 = \chi_{N+1} = 1$, for our matrices must contract to scalars. The reason we discuss this is that ground states, i.e. states that follow area laws can be accurately approximated by MPS with a maximal bond dimension $\chi_{\max}$ [21]. However, the representation in equation (3.62) is far from being unique, so we should choose the representation to be as computationally convenient as possible [21, 7]. For that reason, we will now introduce the canonical form.

First, let us look at the bond between sites n and $n+1$, and consider the bipartition of our Ising chain into sites $L = \{1, 2, \dots, n\}$ and $R = \{n+1, \dots, N\}$. Let X be an invertible

$\chi_{n+1} \times \chi_{n+1}$ matrix, such that

$$M^{[n],j_n} \rightarrow \tilde{M}^{[n],j_n} = M^{[n],j_n} X^{-1}, \quad M^{[n+1],j_{n+1}} \rightarrow \tilde{M}^{[n+1],j_{n+1}} = X M^{[n+1],j_{n+1}}. \quad (3.63)$$

By plugging these new matrices into equation (3.62) we observe they describe the same state $|\psi\rangle$. With this in mind, given a family of matrices $M^{[n],j_n}$, we can decompose these matrices to be $M^{[n],j_n} = \Gamma^{[n],j_n} \tilde{\Lambda}^{[n]}$, where $\{\tilde{\Lambda}^{[n],j_n}\}$ are square, diagonal matrices with positive values $\Lambda_{\alpha+1}^{[n]}$. Then equation (3.62) simply becomes

$$|\psi\rangle = \sum_{\alpha_{n+1}} \Lambda_{\alpha_{n+1}}^{[n+1]} |\alpha_{n+1}\rangle_L \otimes |\alpha_{n+1}\rangle_R, \quad (3.64)$$

where we define

$$|\alpha_{n+1}\rangle_L = \sum_{j_1, \dots, j_n} (M^{[1],j_1} \dots M^{[n],j_n}) |j_1\rangle \otimes \dots \otimes |j_n\rangle, \quad (3.65)$$

$$|\alpha_{n+1}\rangle_R = \sum_{j_{n+1}, \dots, j_N} (M^{[n+1],j_{n+1}} \dots M^{[N],j_N}) |j_{n+1}\rangle \otimes \dots \otimes |j_N\rangle. \quad (3.66)$$

Here we note that due to orthogonality and normalization, we have $\sum_{\alpha_{n+1}} (\Lambda_{\alpha_{n+1}}^{[n+1]})^2 = 1$. Performing this decomposition for each bond yields

$$|\psi\rangle = \sum_{j_1, \dots, j_N} \Lambda^{[1],j_1} \Gamma^{[1],j_1} \Lambda^{[2],j_2} \Gamma^{[2],j_2} \dots \Lambda^{[N],j_N} \Gamma^{[N],j_N} \Lambda^{[N+1],j_{N+1}} |j_1\rangle \otimes \dots \otimes |j_N\rangle, \quad (3.67)$$

where trivially $\Lambda^{[1]} = \Lambda^{[N+1]} = 1$. Typically, we don't perform calculations just like this, but first define the matrices

$$A^{[n],j_n} = \Lambda^{[n]} \Gamma^{[n],j_n} \quad B^{[n],j_n} = \Gamma^{[n],j_n} \Lambda^{[n+1]}.$$

If our MPS is written solely with $A^{[n],j_n}$ matrices then we say our MPS is in left hand form, and if the opposite is true, then the MPS is written in right hand form. The reason why we choose this representation is that this form makes calculating expectation values very easy. We can see this by inserting these matrices into equations (61) and (62)

$$|\alpha_{n+1}\rangle_L = \sum_{\alpha_n, j_n} A_{\alpha_n, \alpha_{n+1}}^{[n],j_n} |\alpha_n\rangle_L \otimes |j_{n+1}\rangle, \quad (3.68)$$

$$|\alpha_n\rangle_R = \sum_{\alpha_n, j_n} B_{\alpha_n, \alpha_{n+1}}^{[n],j_n} |j_n\rangle \otimes |\alpha_{n+1}\rangle_R, \quad (3.69)$$

with orthonormality relation $\langle \alpha_n |_L | \alpha_{n'} \rangle_L = \langle \alpha_n |_R | \alpha_{n'} \rangle_R = \delta_{n,n'}$. To understand this visually, it is good to understand figure 4 where we see how we write our MPS as a tensor network and how partial contractions are enabled by the orthonormality condition and make computing local operators far easier.

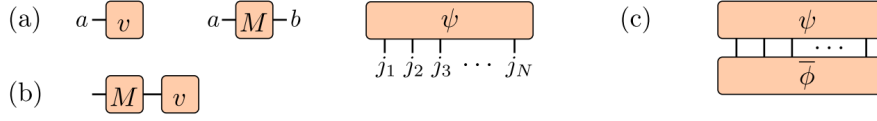


Figure 4: a) Visual representations for a vector v , a matrix M , and the coefficients of a general wave function $|\psi\rangle = \sum_{j_1, \dots, j_N} \psi_{j_1, \dots, j_N} |j_1\rangle \otimes \dots \otimes |j_N\rangle$. b) an example of a partial contraction is a matrix-vector product, with $(Mv)_\alpha = \sum_\beta M_{\alpha\beta} v_\beta$. c) The vector product $\langle\phi||\psi\rangle$ [7]

3.3 TIME EVOLVING BLOCK DECIMATION (TEBD)

Now that we know how to represent states as MPS, we wish to know how to see how they evolve in time with the use of numeric methods. One very powerful tool is the Time Evolving Block Decimation, and concretely, it will allow us to compute

$$|\psi(t)\rangle = U(t)|\psi(0)\rangle. \quad (3.70)$$

Specifically, we will always use a ground state of the system as $|\psi_0\rangle$. This state can be easily obtained using the imaginary time evolution operator $U(\tau) = e^{-\tau H}$ through this limit process

$$|\psi_{\text{GS}}\rangle = \lim_{\tau \rightarrow \infty} \frac{e^{-\tau H} |\psi\rangle}{\|e^{-\tau H} |\psi\rangle\|}. \quad (3.71)$$

What is worthy of note here is the choice of $|\psi\rangle$ does not matter, as this process will minimize the energy of the state for all $|\psi\rangle$. The key ingredient of TEBD is the Suzuki-Trotter approximation, where given two non-commuting operators A and B , the first order Suzuki-Trotter [22, 23] approximation is

$$e^{(A+B)\delta} \approx e^A e^B + o(\delta^2) \quad (3.72)$$

We now make use of the fact that an Ising chain consists of two site operators that act on two neighboring sites to write our Hamiltonian as the sum of two non-commuting operators. In general, this looks like

$$H = \sum_{n \text{ odd}} h^{n,n+1} + \sum_{n \text{ even}} h^{n,n+1} = H_{\text{odd}} + H_{\text{even}}, \quad (3.73)$$

it is important to note that all "odd" h commute with each other, and the same is true for the "even" h . Now, to obtain an equation for our time evolution operator, let δt be a very small time step, and define $U_{\text{even}}(\delta t) = e^{-iH_{\text{even}}\delta t}$ and $U_{\text{odd}}(\delta t) = e^{-iH_{\text{odd}}\delta t}$ respectively. This then gives us the first order Suzuki-Trotter approximation for our time evolution operator

$$U(\delta t) = e^{-iH\delta t} \approx \left(\prod_{n \text{ even}} U^{n,n+1}(\delta t) \right) \left(\prod_{n \text{ odd}} U^{n,n+1}(\delta t) \right). \quad (3.74)$$

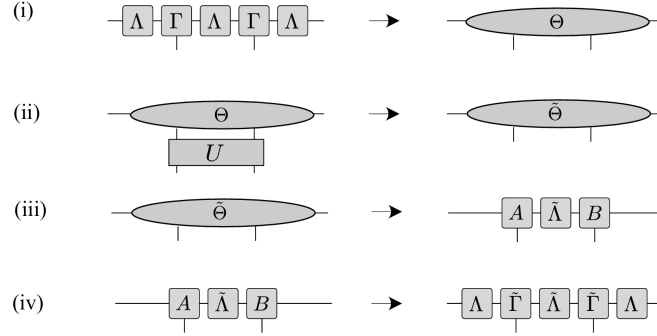


Figure 5: Step-by-step guide to applying the two-gate time-evolution operator and evolving the MPS by one time step.[7]

For reasons which will shortly become apparent, we choose to use the two-site operators $U^{n,n+1}(\delta t) = e^{-ih^{n,n+1}\delta t}$. Now going back to equation (3.67), we now will take a look at how we can apply a local two-site unitary transformation U on our state in the MPS representation. A visual approach is recommended to understand the algorithm, which is provided in figure 5. First take two sites n and $n+1$. We will now write $|\psi\rangle$ in the $\{|\alpha_n\rangle_L \otimes |j_n\rangle \otimes |j_{n+1}\rangle \otimes |\alpha_{n+2}\rangle_R\}$ basis as follows:

$$|\psi\rangle = \sum_{\alpha_n, j_n, j_{n+1}, \alpha_{n+2}} \Theta_{\alpha_n, \alpha_{n+2}}^{j_n, j_{n+1}} |\alpha_n\rangle \otimes |j_n\rangle \otimes |j_{n+1}\rangle \otimes |\alpha_{n+2}\rangle, \quad (3.75)$$

where Θ is given by equation (3.67)

$$\Theta_{\alpha_n, \alpha_{n+2}}^{j_n, j_{n+1}} = \sum_{\alpha_{n+1}} \Lambda_{\alpha_n}^{[n]} \Gamma_{\alpha_n, \alpha_{n+1}}^{[n], j_n} \Lambda_{\alpha_{n+1}}^{[n+1]} \Gamma_{\alpha_{n+1}, \alpha_{n+2}}^{[n+1], j_{n+1}} \Lambda_{\alpha_{n+2}}^{[n+2]}. \quad (3.76)$$

Now we see applying the unitary is very simple, we just perform a sum over our j indices

$$\tilde{\Theta}_{\alpha_n, \alpha_{n+2}}^{j_n, j_{n+1}} = \sum_{j'_n, j'_{n+1}} U_{\alpha_n, \alpha_{n+2}}^{j'_n, j'_{n+1}} \Theta_{\alpha_n, \alpha_{n+2}}^{j'_n, j'_{n+1}}. \quad (3.77)$$

Now, just as before, we must extract our $\tilde{A}^{[n]}$, $\tilde{B}^{[n+1]}$ and $\tilde{\Lambda}^{[n+1]}$. To perform the singular value decomposition, we first reshape our tensor by merging indices j_n with α_n and j_{n+1} with α_{n+2} . This yields us a matrix $\tilde{\Theta}_{j_n \alpha_n, j_{n+1} \alpha_{n+2}} \in \mathbb{C}^{d_{\chi_n} \times d_{\chi_{n+2}}}$

$$\tilde{\Theta}_{j_n \alpha_n, j_{n+1} \alpha_{n+2}} = \sum_{\alpha_{n+1}} \tilde{A}_{j_n \alpha_n, \alpha_{n+1}}^{[n]} \tilde{\Lambda}_{\alpha_{n+1}, \alpha_{n+1}}^{[n+1]} \tilde{B}_{\alpha_{n+1}, j_{n+1} \alpha_{n+2}}^{[n+1]}. \quad (3.78)$$

We note now that, just like before, our MPS is now in mixed canonical form. In this last step, all we must do is define our new $\tilde{\Gamma}$ as follows to bring it back into right canonical form:

$$\tilde{\Gamma}_{\alpha_n, \alpha_{n+1}}^{[n], j_n} = \left(\Lambda_{\alpha_n}^{[n]} \right)^{-1} \tilde{A}_{j_n \alpha_n, \alpha_{n+1}}^{[n]} \quad \tilde{\Gamma}_{\alpha_{n+1}, \alpha_{n+2}}^{[n+1], j_{n+1}} = \tilde{B}_{\alpha_{n+1}, j_{n+1} \alpha_{n+2}}^{[n+1]} \left(\Lambda_{\alpha_{n+2}}^{[n+2]} \right)^{-1}. \quad (3.79)$$

In practice, however, we would typically use the second order Suzuki-Trotter decomposition [24, 25].

$$e^{-iH\delta t} \approx e^{-iH_{\text{even}} \frac{\delta t}{2}} e^{-iH_{\text{odd}} \delta t} e^{-iH_{\text{even}} \frac{\delta t}{2}} + O(\delta t^3). \quad (3.80)$$

In conclusion, a TEBD simulation is specified by the Trotter step δt , a maximum bond dimension χ_{max} (or truncation tolerance $\varepsilon_{\text{trunc}}$), and the sweep order (even/odd). We see after one time step, the bond dimension at the bond between sites n and $n+1$ grew to $d\chi$, so we conclude that entanglement entropy grows exponentially with time. This severely limits the maximal simulation time, so to access larger $T_{\text{max},2}$, one must (partly) sacrifice accuracy. Accuracy is controlled by simultaneously decreasing δt (reduces Trotter error) and increasing χ_{max} (reduces truncation error).

3.4 LINEAR PREDICTION

Time evolution with MPS is limited to a maximum reliable time $T_{\text{max},1}$. Linear prediction is a simple way to extrapolate a complex time series beyond $T_{\text{max},1}$ so that the Fourier spectrum is sharper [8, 9]. Given a time series $\{x_{t_1}, \dots, x_{t_n}\}$, where $t_n \equiv T_{\text{max},1}$, the ansatz behind linear prediction is that the next time step is essentially a linear combination of previous time steps. Mathematically this can be written as

$$\tilde{x}_{t_{n+1}} = - \sum_{i=1}^p a_i x_{t_{n+1-i}}. \quad (3.81)$$

The coefficients a_i are determined by minimizing the least square error

$$E = \sum_{i>p} \frac{|\tilde{x}_{t_i} - x_{t_i}|^2}{w_n}, \quad (3.82)$$

where x_{t_i} are our true values, $\tilde{x}_{t_i}$ are the values produced by our ansatz and w_n act as weights. Minimizing the error in equation (3.82) can't be done directly by merely formulating equation (3.81) as a matrix vector multiplication problem directly, of the form

$$X\mathbf{a} = -\mathbf{x}, \quad (3.83)$$

where the vector $\mathbf{x}$ contains all values x_{t_i} with $i > p$ and our matrix X has the time-series values corresponding to the values we wish to predict as rows, so for instance the first row would be $(x_{t_1} \dots x_{t_p})$, the second one would be $(x_{t_2} \dots x_{t_{p+1}})$ and so on and so forth, since X isn't a square matrix and therefore can't be inverted.

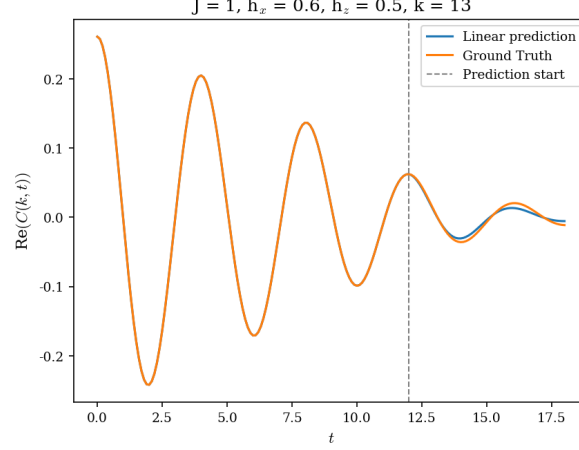


Figure 6: An example of $C(k, t)$, where after 120 time steps of size $\delta t = 0.1$, we employ linear prediction to extrapolate the last 60 timesteps. The absolute difference - as discussed at the end of this section - is 41.135.

Instead we take a detour by defining the matrix R and vector $\mathbf{r}$ with

$$R_{ij} = \sum_{n>p} \frac{x_{n-j}^* x_{n-i}}{w_n}, \quad r_j = \sum_{n>p} \frac{x_{n-j}^* x_n}{w_n}$$

and obtaining the following set of linear equations:

$$R\mathbf{a} = -\mathbf{r}, \quad (3.84)$$

which is solved by inverting R

$$\mathbf{a} = -(R)^{-1}\mathbf{r}, \quad (3.85)$$

where R is a positive definite matrix for positive weights w_n .

That being said, it is not immediately clear as to why this ansatz should work for very large times, so some justification is in order. It can be demonstrated that LP generates a superposition of oscillating and decaying terms, which as we will shortly see matches our correlators in k -space quite well. We will not dive deeper into this, as the intricacies behind LP go beyond the scope of this thesis. To do this, we employ the linear prediction function of TenPy, a Python library designed to calculate such functions with the help of tensor networks. Figure 6 depicts an example of linear prediction being used to simulate our $C(k, t)$ beyond $T_{\max,1}$.

Since this is a reliable, well-tested method, we shall use it as a benchmark to judge our neural networks ability to learn these patterns. To judge accuracy, we shall create two "copies" of $C(k, t)$ where after $T_{\max,1}$ one copy $C(k, t)$ will be the values obtained through TEBD, which we will treat as the ground truth, and the second copy $\tilde{C}(k, t)$ will be obtained through linear

prediction, and we will judge the relative accuracy by calculating

$$\sum_{k,t} |\tilde{C}(k,t) - C(k,t)|. \quad (3.86)$$

3.5 TEMPORAL CONVOLUTIONAL NETWORKS (TCN)

Historically, when dealing with sequential tasks such as time series extrapolation, one would use a recurrent neural network or a variant thereof such as an LSTM (Long Short-Term Memory) to produce the outputs. While this approach is valid in certain cases, these sequential types are known to struggle with capturing patterns over long periods of time. In this thesis, we explore a relatively new neural network architecture called temporal convolutional networks, which has been shown to outperform its sequential predecessors on various tasks. In order to give machine learning a chance at outperforming TenPy's linear prediction function, we will implement the TCN written by Bai et al.[10], in hopes of recreating their success in this time series forecasting task.

But suppose we have a time series $\{x_0, \dots, x_n\}$ and we want to predict $\{y_0, \dots, y_n\}$. On the most fundamental level, our TCN can be represented by the function

$$f(x_0, \dots, x_n) = (y_0, \dots, y_n), \quad (3.87)$$

where y_i depends only on $\{x_0, \dots, x_i\}$. As a small aside, it may seem that we are restricted to choosing $\{y_0, \dots, y_n\} = \{x_{n+1}, \dots, x_{2n}\}$. However, when training the network, one can truncate the final output of the network to only contain the last m results, and through training the TCN then learns that the last m values are $\{x_{n+1}, \dots, x_{n+m}\}$, but for simplicity's sake we will refer to the output as $\{y_0, \dots, y_n\}$ in this section. The goal of our neural network is to minimize the loss function

$$L(f(x_0, \dots, x_n), y_0, \dots, y_n),$$

and we will give our TCN data from many different Ising chains with their own unique parameters in hopes of our network learning to generalize across various systems. But now let us explore the TCN architecture. TCN's differentiate themselves from other neural networks through three architectural features: Causal convolutions, dilation layers and residual blocks. Figure 7 visualizes these concepts.

Causal convolutions TCNs follow a 1D fully-convolutional network architecture as demonstrated in figure 7 (a), with each hidden layer as long as the input layer. To keep the lengths equal, we add zero-padding of size (kernel-size - 1). This zero-padding, as we see in figure 7 (a) also allows us to make sure an output at time t is the result of a convolution involving inputs at the same time and earlier.

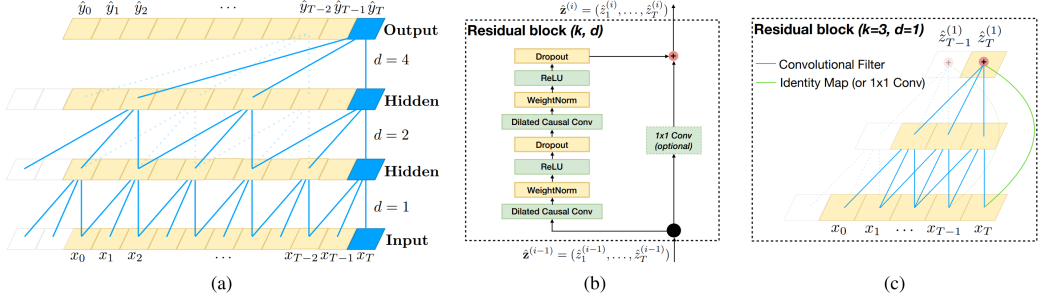


Figure 7: a) by having multiple dilation layers, the receptive field is large enough to capture all inputs. b) An 1x1 convolution is applied within the residual block if the dimensions of the output and input don't match. c) The green line depicts the Identity map or 1x1 Convolution.[10]

Dilation layers That being said, as of now, the receptive field, i.e. the number of time-steps our network can look back only grows linearly with the depth of the network. For that reason we introduce the concept of dilation layers which makes the receptive field grow exponentially in proportion of the depth of the neural network. More concretely, given a sequence $\mathbf{x} \in \mathbb{R}^n$, we define a kernel $f : \{0, \dots, k-1\} \rightarrow \mathbb{R}$ and the dilation factor $d = 2^i$ with i being representing the layer being convoluted, the dilated convolution function $\mathcal{F}(m)$ producing output at position m is defined as

$$\mathcal{F}(m) = (\mathbf{x} *_d f)(m) = \sum_{j=0}^{k-1} f(j)x_{s-j*d} \quad (3.88)$$

Coming back to figure 7 (a), we see how by increasing the networks depth we enable it to look further and further into the past.

Residual blocks Residual blocks incorporate the former two features and are constructed as demonstrated in figure 7 (b) and (c). After feeding the input into the main path, we add the input to the transformed output before applying the activation function and feeding the output to the next residual block, essentially

$$o = \text{Activation}(x + \mathcal{M}(x)). \quad (3.89)$$

We do this because it has been repeatedly shown that this method of allowing the network to learn modifications to the input sequence instead of the entire transformation leads to more accurate outputs.

We adopt the Bai et al. configuration: Causal dilated blocks without temporal pooling, kernel size k , dilations doubling per layer, residual connections, and left padding $(k-1)d$ per layer so that the shape stays (batch, channels, time) throughout.

4

Benchmarks

This section explains the full pipeline (TEBD ground truth $\rightarrow$ Linear Prediction baseline $\rightarrow$ TCN forecaster), the precise network structure, the data splits, and the tests we run.

4.1 OVERALL PIPELINE

We simulate the transverse-field Ising chain (TFIC) with open boundaries (OBC) at $J = 1$ using TenPy’s TEBD function, generate a local quench at the chain center of the ground state with σ^y , and measure $C(k, t)$. The calculations were done using the Tenpy library (Version 1.0.7)[26]. TEBD up to $T_{\max,1}$ provides the ground truth. Beyond $T_{\max,1}$ we compare (i) TenPy’s Linear Prediction (LP) extrapolation and (ii) a Temporal Convolutional Network (TCN) forecast trained on windows entirely inside $[0, T_{\max,1}]$. This pipeline is also shown in figure 8.

The fundamental difference between the two methods is in the case of Linear prediction, we extrapolate the two-point correlator by solving a least-square minimization problem for

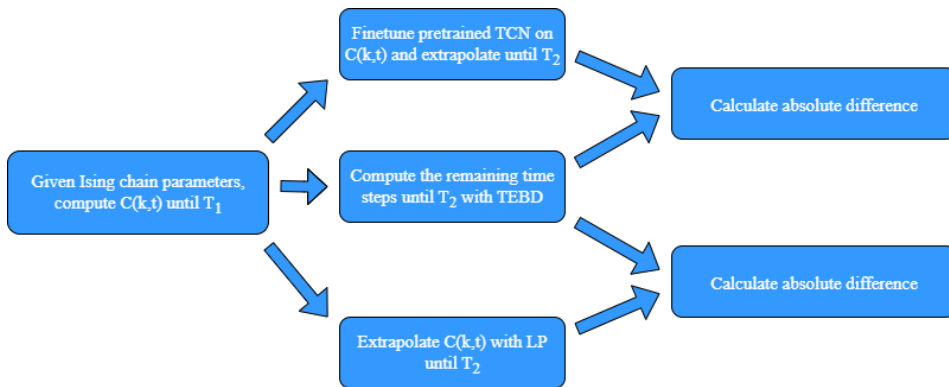


Figure 8: Test setup visualized as a flowchart.

each k term individually, whereas in the case of the TCN we train it on a plethora of Datasets, then finetune it on all available time steps of $C(k, t)$ and then we extrapolate.

Finite-size sanity. We use $N = 90$ and $T_{\max,2} = 18$ with $J = 1$, which more than satisfies the no-reflection bound

$$N \geq J_S \min(1, 2 \frac{h_{x,S}}{J_S}) T_{\max,2} \quad (4.90)$$

for all points we study. For these Benchmark tests we use $\delta t = 0.1$ to stay consistent.

4.2 DATASETS AND SPLITS

We consider 23 TFIC parameter points (h_x, h_z) covering both phases and weak/strong breaking of integrability via $h_z > 0$, see Tables 4.1 and 4.2. For each (h_x, h_z) and each momentum k we obtain the complex series $\tilde{C}(k, t_n)$ with $t_n = n \delta t$.

Table 4.1: Parameters of the 15 TFIC datasets used in tests 1-3 (see section 4.5). Transverse field h_x and longitudinal field h_z (with $J = 1$).

ID	h_x	h_z
1	0.3	0.0
2	0.5	0.0
3	0.7	0.0
4	1.0	0.0
5	1.5	0.0
6	2.0	0.0
7	2.5	0.0
8	0.9	0.1
9	1.0	0.1
10	1.2	0.1
11	0.6	0.5
12	0.9	0.5
13	1.2	0.5
14	0.3	1.2
15	1.5	1.2

To generate our batches, for each k (two channels: real/imag) we form input windows of length $R_{\text{win}} = 60$, and pair them with the next 45 target steps. When training the TCN, we do an 80-20 split for training and validation. We also note that Pytorch currently only operates with real numbers, so for the sake of convenience we choose to interleave the real and imaginary parts, and shape our Datasets to look like $(\text{Re}(\tilde{C}(k_1, t)), \text{Im}(\tilde{C}(k_1, t)), \text{Re}(\tilde{C}(k_2, t)), \text{Im}(\tilde{C}(k_2, t)), \dots)$ for all t .

During training and fine-tuning, we only train the TCN to predict the last 15 steps by first predicting the 15 time steps, appending them to the sequence and discarding the last 15 time steps and doing this until we have forecasted 45 time steps. We then calculate the loss function

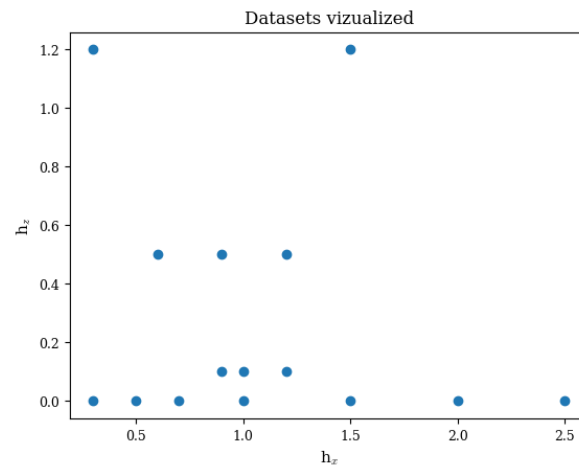


Figure 9: Datasets from Table 4.1 visualized in parameter space.

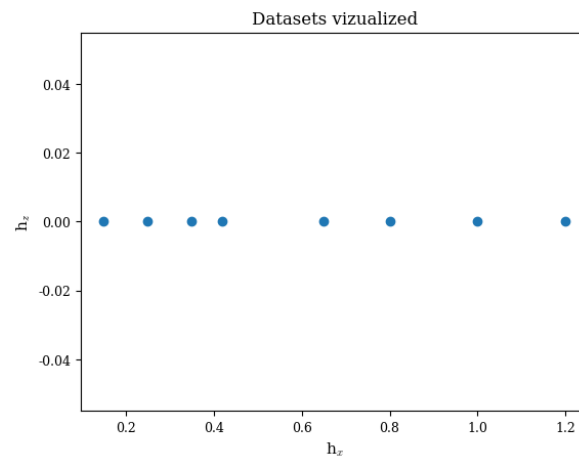


Figure 10: Datasets from Table 4.2 visualized in parameter space.

Table 4.2: Parameters of the 8 additional TFIC datasets used in tasks 4-7. Transverse field h_x and longitudinal field h_z (with $J = 1$).

ID	h_x	h_z
16	0.15	0.0
17	0.25	0.0
18	0.35	0.0
19	0.45	0.0
20	0.65	0.0
21	0.8	0.0
22	1	0.0
23	1.2	0.0

and update the weights. We do this to train the network to simulate the prediction step and become tolerant to its own faulty outputs. We use this training protocol during training as well as fine tuning.

4.3 MODEL STRUCTURE OF OUR TCN

Stack and receptive field. We use $L = 4$ blocks with channel widths $[64, 64, 64, 128]$ and doubling dilations $d \in \{1, 2, 4, 8\}$. With kernel size $K = 7$ the receptive field is

$$R = (K - 1)(2^L - 1) + 1 = (7 - 1)(2^4 - 1) + 1 = 91, \quad (4.91)$$

so we choose the input window $R_{\text{win}} \leq R$ and keep only the last m time indices of the full-length prediction (the ones with full context). A final 1×1 head maps $128 \rightarrow 2$ channels (real/imag).

Loss and optimization. The default loss is MSE over the forecast horizon and both channels, using the Adam optimizer. At evaluation we do a sliding-window, just like in training, to predict the remaining 60 time steps. For both training and fine-tuning, we have implemented an early-stop feature with patience = 5 epochs that stores the best model, so our results are reproducible regardless of the initial seed. The learning rate in training starts at 10^{-3} , and in fine-tuning at 10^{-7} .

4.4 BASELINES

We implement Linear Prediction (LP) as a weighted least-squares $\text{AR}(p)$ fit on $C(k, t)$ windows entirely inside $[0, T_{\text{max},1}]$ and extrapolate m steps (see [8, 9] for details). The LP per-dataset errors are listed in table 4.3.

As previously stated, we will be using equation (3.86) to compare our results and determine the quality of our numerical predictions.

Table 4.3: Linear Prediction (LP) baseline error per dataset. Metric: absolute difference $\sum_{k,t} |\tilde{C}(k,t) - C(k,t)|$.

Dataset	LP abs. diff.
1	10.987
2	17.895
3	32.096
4	33.714
5	33.966
6	34.015
7	34.026
8	31.877
9	32.686
10	33.626
11	41.135
12	33.977
13	30.665
14	34.240
15	35.378

4.5 EXTRAPOLATION TASKS

We organize seven extrapolation tasks for our TCN to accomplish that probe in-regime performance and generalization across phases and integrability.

1. **SOLO + fine-tuning.** Train on 14 datasets, fine-tune on the first part (first 120 steps) of the held-out series, then forecast beyond $T_{\max,1}$. This is the best case use of limited ground truth.
2. **Train on integrable ($h_z = 0$), test on non-integrable ($h_z > 0$).**
3. **Train on non-integrable, test on integrable.**
4. **Train on paramagnets ($h_x > \frac{1}{2}$), test on ferromagnets ($h_x < \frac{1}{2}$).**
5. **Train on ferromagnets, test on paramagnets.**
6. **Train on ferromagnets, test on ferromagnets (Analogous to test 1).**
7. **Train on paramagnets, test on paramagnets (Analogous to test 1).**

For each test, we report the per-dataset E in tables (TCN vs. LP) and include representative plots with TEBD (truth), LP, and TCN overlaid beyond $T_{\max,1}$. We also note, since linear prediction is an excellent tool for extrapolating such time series, and that all systems consist of 90 lattice sites and are simulated for the same total time, we shall not normalize the errors for clarity's sake.

5

Results

5.1 GENERALIZING ACROSS VARIOUS ISING CHAINS

Table 5.1: Task 1: SOLO + fine-tuning

ID	h_x	h_z	Abs. diff
1	0.3	0.0	14.487
2	0.5	0.0	13.993
3	0.7	0.0	26.273
4	1.0	0.0	10.913
5	1.5	0.0	13.786
6	2.0	0.0	20.180
7	2.5	0.0	49.381
8	0.9	0.1	17.355
9	1.0	0.1	14.315
10	1.2	0.1	9.914
11	0.6	0.5	47.628
12	0.9	0.5	24.916
13	1.2	0.5	20.159
14	0.3	1.2	10.235
15	1.5	1.2	15.447

Figure 11 shows us a comparison of the performance between linear prediction and TCN. We see that our TCN that has been trained on all datasets except one using all 180 time steps, then fine-tuned on the first 120 "given" time steps of the remaining dataset outperforms linear prediction by a good margin on twelve of our samples. Figures 12 (a) and (b) showcase samples of the best and worst results in this test. One observation we make is that the cases linear prediction won, datasets 1, 7 and 11, are "edge points" as demonstrated in figure 9, because they have less neighboring datasets in the $h_x - h_z$ parameter space, with the largest error being exhibited by dataset 7 ($h_x = 2.5, h_z = 0$), which in figure 9 has the greatest distance

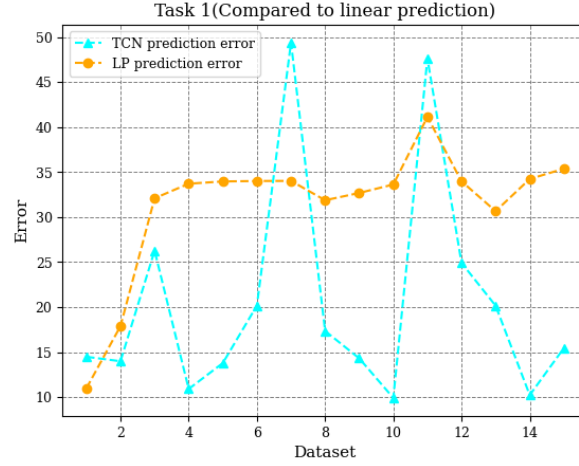


Figure 11: Graph depicting the errors of linear prediction and our TCN for each dataset

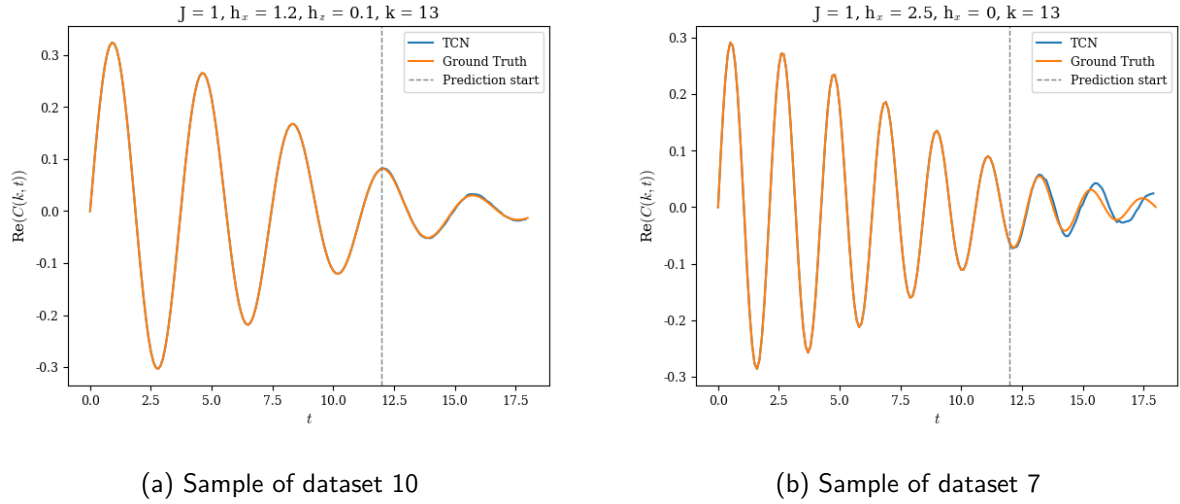


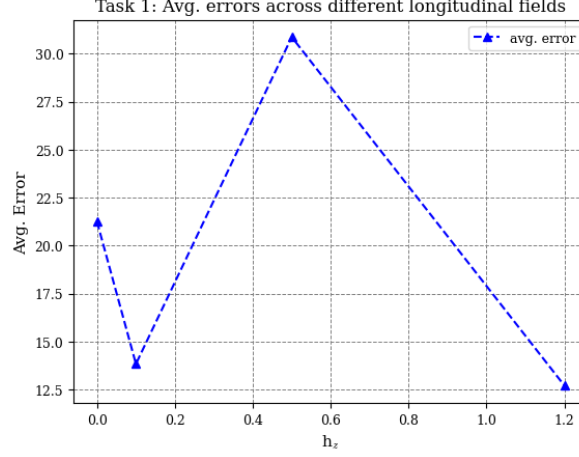
Figure 12: (a) Sample of Dataset 10, which delivered the smallest deviation from the ground truth, and (b) Sample of Dataset 7, which delivered the greatest deviation from the ground truth. Extrapolation starts at time step 120.

from any neighbors.

An interpolation-based argument does not suffice here, since other edge-cases such as Datasets 14 and 15, by that logic, should perform similarly to Dataset 7. This is not the case, however, as TCN delivers results that are three and two times better than linear prediction respectively. Additionally, integrability also doesn't seem to have much of an effect. If it had, we would expect our TCN to perform worse on the non-integrable systems, but we simply see no correlation, as some non-integrable systems deliver bad results, whereas others even beat most TCN outputs on integrable cases. We verify this by looking at the average errors for different values of h_z in table 5.2 as well as figure 13.

Table 5.2: average errors of TCN as a function of h_z (Task 1)

h_z	0	0.1	0.5	1.2
$E_{\text{avg.}}$	21.25	13.3	30.87	12.7

Figure 13: Graph depicting no relation between h_z and the average error of the TCN prediction in test 1

5.2 EFFECTS OF INTEGRABILITY AND LACK THEREOF

Table 5.3: Task 2: Train on integrable, test on non-integrable

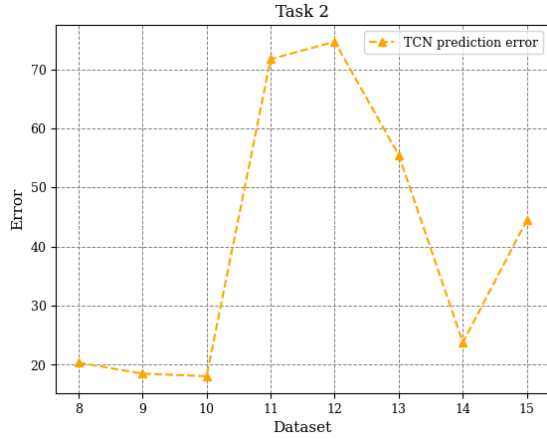
ID	h_x	h_z	Abs. diff
8	0.9	0.1	20.359
9	1.0	0.1	18.479
10	1.2	0.1	18.048
11	0.6	0.5	71.699
12	0.9	0.5	74.649
13	1.2	0.5	55.472
14	0.3	1.2	23.764
15	1.5	1.2	44.458

We now perform similar simulations, only this time we trained our TCN on integrable and non-integrable systems, to see how well it would generalize on their counterparts. The results are displayed in tables 5.3 and 5.4 as well as figures 14 (a) and (b). We notice a significant drop in the quality of our predictions, which will more or less be noticeable in the next tests, so the focus will be on comparing the relative compatibility of our TCN with different regimes.

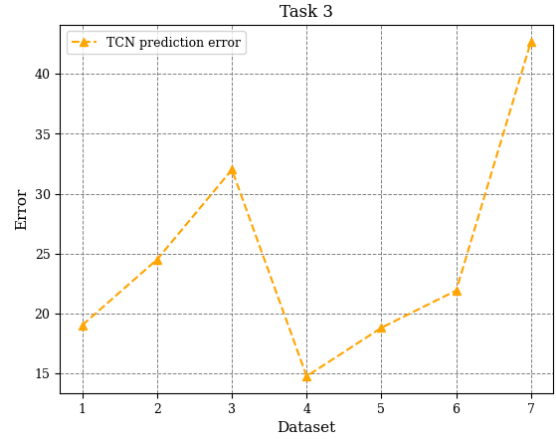
We see when we trained on the integrable systems, our TCN performed well only on Datasets 8, 9, 10 and 14. It makes sense for our TCN to perform well on Datasets 8, 9 and 10 due to their close proximity to the critical point, and this may be a case of interpolation, and as for Dataset 14, we have seen previously that our TCN tends to perform quite well for lower

Table 5.4: Task 3: Train on non-integrable, test on integrable

ID	h_x	h_z	Abs. diff
1	0.3	0.0	19.031
2	0.5	0.0	24.510
3	0.7	0.0	32.007
4	1.0	0.0	14.760
5	1.5	0.0	18.807
6	2.0	0.0	21.913
7	2.5	0.0	42.683



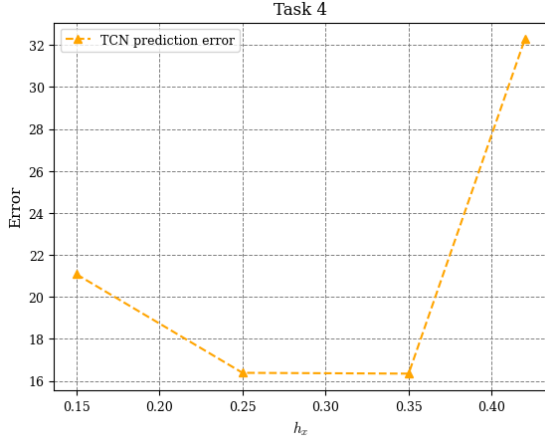
(a) Errors for task 2



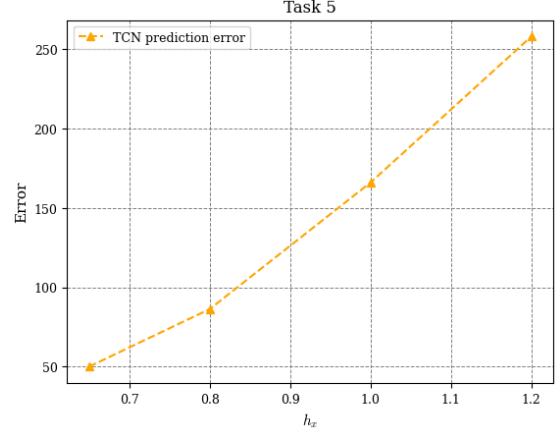
(b) Errors for task 3

Figure 14: (a) Training on integrable systems and testing on non-integrable ones, and (b) training on non-integrable systems and testing on integrable ones.

transversal fields. But in general, the average error across these simulation is $E_{\text{avg.}} = 40.87$. In contrast, when we train on the non-integrable systems to then extrapolate the two-point correlator of the integrable Ising chains, we obtain an average error of $E_{\text{avg.}} = 24.82$, which is nearly twice as good. A plausible argumentation for this is that when a system is integrable, as stated in section 2.2, relaxes to the GGE, and exhibits less chaotic dynamics, whereas when the TCN is trained on Data of "lesser complexity", it struggles with generalizing and forecasting more complex time series. The only exception we have here is Dataset 7, the transverse-field Ising chain with the largest transversal field. TCN consistently performed bad on this dataset, and the reason might be that, just as argued in section 5.1, this is too far away in the " $h_x - h_z$ parameter space" from other neighboring Datasets in figure 9, so the TCN cannot use interpolation to determine the correct dynamics even when fine-tuned on the first 120 steps.



(a) Errors for task 4



(b) Errors for task 5

Figure 15: (a) Probing the TCN's ability to generalize after being trained in the paramagnetic regime, and (b) Probing the TCN's ability to generalize after being trained in the ferromagnetic regime.

5.3 GENERALIZING ACROSS DIFFERENT PHASES

In these tests, we train solely on Ising chains with $h_x > \frac{1}{2}$ and $h_z = 0$, and $h_x < \frac{1}{2}$ and $h_z = 0$ respectively (see figure 10 and table 4.2), to then explore the networks ability to adapt to systems exhibiting different dynamics. The results are visible in tables 5.5, 5.6 and figures 15 (a) and (b).

Table 5.5: Task 4: Train on paramagnets, test on ferromagnets

ID	h_x	h_z	Abs. diff
1	0.15	0.0	21.082
2	0.25	0.0	16.385
3	0.35	0.0	16.345
4	0.42	0.0	32.280

Table 5.6: Task 5: Train on ferromagnets, test on paramagnets

ID	h_x	h_z	Abs. diff
4	0.65	0.0	50.030
5	0.8	0.0	86.291
6	1	0.0	165.974
7	1.2	0.0	257.936

From figure 15 (a) we see that our TCN generalizes way better when trained in the paramagnetic regime than in the ferromagnetic regime. One might argue that the TCN simply performs better in the ferromagnetic domain. The dynamics of paramagnets are theoretically simpler than those of ferromagnets, so this result was not expected. We note this is a fair comparison, as in both cases, the TCN was trained on four datasets.

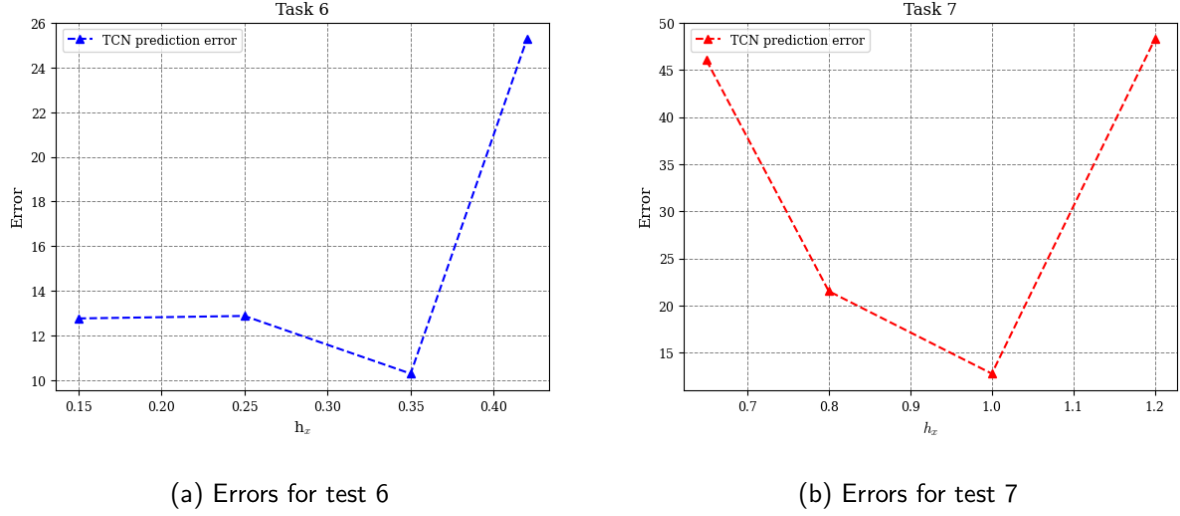


Figure 16: Training and testing on (a) ferromagnets and (b) paramagnets.

From these results, one could hypothesize that the paramagnetic regime offers our TCN a wider range of patterns which enables the TCN to better generalize on different Ising chains. A possible explanation is that if we look at figures 12 (a) and (b), we see that the frequency with which $C(k, t)$ oscillates grows with the magnitude of the transverse field h_x , so the TCN is better at forecasting $C(k, t)$ with frequencies lower than those that were used to train the TCN.

Additionally, we notice in figure 15 (a) that the TCN's error drastically increases when reaching $h_x = 0.42$, despite being the closest to the paramagnetic regime. From this we can theorize that perhaps the dynamics of the TFIC become harder to predict the closer we get to the critical point.

5.4 GENERALIZING WITHIN THE SAME PHASE

Now we want to explore the network's ability to not only generalize, but also learn and apply patterns within the same phase. As of yet, our results suggest our TCN is better at predicting the two-point correlator of ferromagnets, which is an unexpected result, since perturbations in ferromagnets behave like two quasi-particles (domain walls), whereas in paramagnets they are spin flips and behave like one quasi-particle, but may be explained by the "frequency correlates with h_x " argument we made in the last section. For that reason, in the following tests we will test the TCN's ability to function within the same phase.

We see that these results are consistent with what we have previously noticed, since for starters, the errors of the TCN prediction drastically increase around the critical point. This is ex-

Table 5.7: Test 6: Train on three ferromagnets, test on remaining ferromagnet

ID	h_x	h_z	Abs. diff
16	0.15	0.0	12.405
17	0.25	0.0	12.873
18	0.35	0.0	10.286
19	0.42	0.0	25.279

Table 5.8: Test 7: Train on three paramagnets, test on remaining paramagnet

ID	h_x	h_z	Abs. diff
20	0.65	0.0	46.023
21	0.8	0.0	21.539
22	1	0.0	12.793
23	1.2	0.0	48.258

pected in prediction models, as the two-point correlator behaves more erratically near the critical point. Additionally, three of the edge points in figures 16 (a) and (b) are edge points, with the exception of the TFIC with $h_x = 0.15$. This can be explained by the fact that h_x is lower than the transverse fields of the TFICs we used for training. Additionally, the prediction errors corresponding to the middle points in both figures are very good, which further affirms the TCN’s interpolation ability for time series forecasting. It also performs significantly better in the ferromagnetic phase, which supports the findings of the previous section.

5.5 PROBING THE LIMITS OF TCN

In this final task, we wish to illustrate the limits of our TCN’s predictive ability using all the conclusions we have gathered on what enables our TCN to act as a reliable extrapolator for a two-point correlator. To do so we first retrain our TCN as we did in the first benchmark test, and use only the first 105 time steps, fine-tuning included. Second, we trained the TCN to forecast the two-point correlator of dataset 23 as we did in the fifth task. In both cases, we extrapolate $C(k, t)$ until $T_{\max=40}$ to then compute the spectral function.

What sets these two test runs apart is that

1. in contrast to dataset 23, for dataset 4 we leverage the TCN’s interpolation ability and
2. the transverse field of dataset 4 is lower than that of dataset 23.

The results are displayed in figures 17 (a) and (b), and to analyze the results, we also plotted $\epsilon(k)$ from equation (2.30) to see how well our TCN performs.

As expected, the prediction showcased in figure 17 (a) matches the theoretical result, and allowed us to accurately calculate the spectral function with high accuracy. That being said,

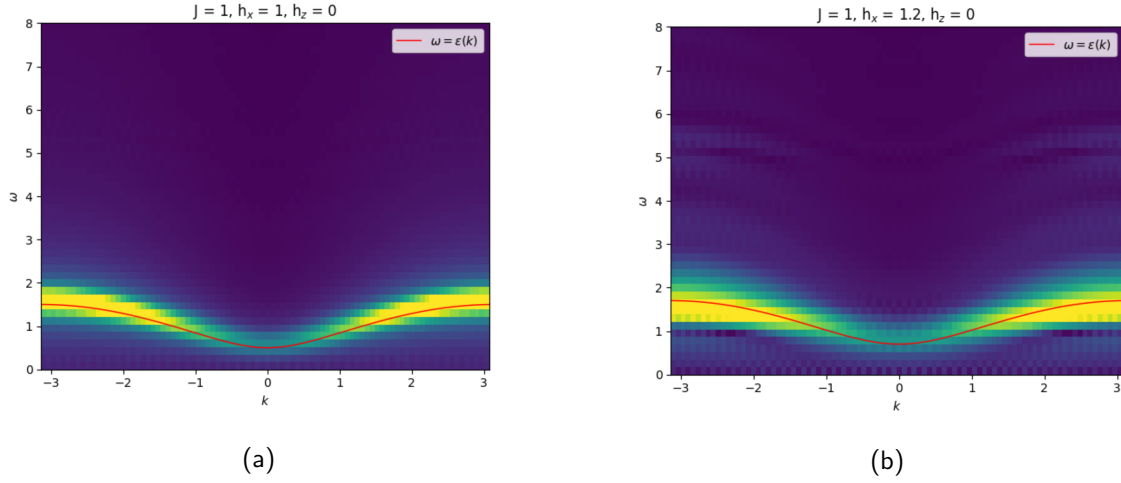


Figure 17: Spectral function $|S(k, \omega)|$ of (a) dataset 4 and (b) dataset 23, showcasing the conditions required for a TCN to make sensible predictions.

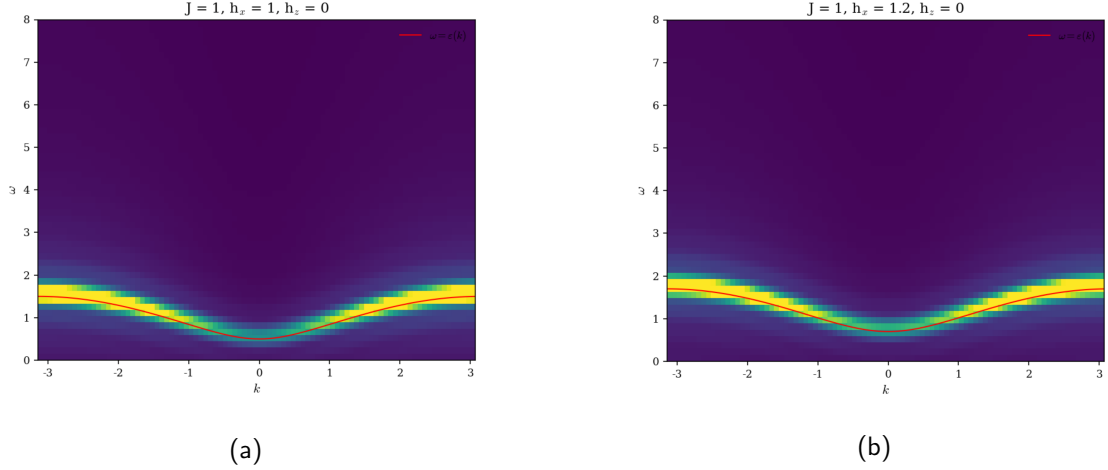


Figure 18: Spectral function $|S(k, \omega)|$ calculated as in figures 17 (a) and (b) by extrapolating $C(k, t)$ with the use of linear prediction.

the extrapolated Fourier-transformed two-point correlator in figure 17 (b) is noticeably wrong, since the spectral ridge deviates from $\epsilon(k)$ near $k = \pm\pi$ and there are multiple bright areas far above the theory curve which shouldn't exist.

Additionally, it is also noteworthy to mention, that while our TCN managed to outperform linear prediction in many instances in the first task, both methods are very good at forecasting two point correlators of quantum Ising chains as is seen in figures 18 (a) and (b), which depict the same spectral functions as in figures 17 (a) and (b), but obtained through linear prediction. We see that given the right training data, our TCN can extrapolate $C(k, t)$ far better than linear prediction, but when computing the spectral function, both methods deliver qualitatively similar spectral functions.

5.6 CONCLUSION

The purpose of this thesis is to find a machine-learning based method to extrapolate observables. We investigated whether a Temporal Convolutional Network (TCN) can extrapolate the two-point correlator $C(k, t)$ of the quantum Ising chain beyond the TEBD window, using Linear Prediction (LP) as a strong baseline. The ground truth was computed up to $T_{\max,2}$, and models were trained on two-point correlators of quantum Ising chains, then fine-tuned on the chosen two-point correlators values that lie in $[0, T_{\max,1}]$ and evaluated on the held-out tail. Our main findings were:

1. *In-regime performance.* With leave-one-out training plus short, budgeted fine-tuning on the first 120 time steps, the TCN outperforms LP on 12/15 datasets (largest gain on dataset 10, weakest on dataset 7).
2. *Generalization.* Errors typically increase with the Euclidean distance in parameter space (h_x, h_z) from training to test, and high h_x (higher oscillation frequency) is particularly challenging for both TCN and LP.
3. *Integrability.* We find no consistent monotonic dependence on h_z (integrability breaking) in the leave-one-out setting; integrability alone does not explain performance.
4. *Asymmetry across regimes.* Training on non-integrable and paramagnetic points generalizes better to integrable and ferromagnetic points respectively than the reverse.

On the level of the spectral function $S(k, w)$ we find that, whenever the TCN manages to accurately extrapolate $C(k, t)$, its $S(k, w)$ are qualitatively similar to those obtained from LP. In the ‘hard’ cases with large h_x and poor interpolation support, the breakdown is clearly visible in $S(k, w)$ as spectral weight that drifts away from the analytic dispersion $\epsilon(k)$ and develops additional bright bands where none should exist.

Overall, these initial results show that TCNs are a promising tool for forecasting dynamical correlators of quantum spin chains, provided that they are trained on sufficiently diverse and phase-aware datasets. This is useful, as it enables calculating $S(k, w)$ reliably with modest computational resources, which is important for simulating e.g. inelastic neutron scattering experiments with high accuracy. As a next step, it would be natural to test the robustness of this architecture on larger and more complex models (e.g. Heisenberg chains or ladders) and to probe longer extrapolation times. It would also be interesting to quantify how the required amount and coverage of training data scales with system size and model complexity, and whether similar architectures can accelerate the numerical exploration of many-body dynamics more broadly.

Bibliography

- [1] M. Greiner, O. Mandel, T. Esslinger, T. W. Hänsch, and I. Bloch, “Quantum phase transition from a superfluid to a mott insulator in a gas of ultracold atoms,” 2002. cited on p. [1](#)
- [2] I. Bloch, J. Dalibard, and W. Zwerger, “Many-body physics with ultracold gases,” 2008. cited on p. [1](#)
- [3] C. Monroe, W. C. Campbell, L.-M. Duan, M. Gong, A. V. Gorshkov, P. W. Hess, R. Islam, J. Kim, G. Pagano, P. Richerme, C. Senko, and N. Y. Yao, “Programmable quantum simulations of spin systems with trapped ions,” 2021. cited on p. [1](#)
- [4] B. Lake, “Quantum criticality and universal scaling of a quantum antiferromagnet,” 2005. cited on p. [1](#)
- [5] P. Pfeuty, “The one-dimensional ising model with a transverse field,” 1970. cited on p. [1](#), [2](#), [3](#)
- [6] M. Rigol, V. Dunjko, V. Yurovsky, and M. Olshanii, “Relaxation in a completely integrable many-body quantum system,” 2007. cited on p. [1](#), [9](#)
- [7] J. Hauschild and F. Pollmann, “Efficient numerical simulations with tensor networks: Tensor network python (tenpy),” 2018. cited on p. [2](#), [13](#), [14](#), [15](#), [17](#), [18](#)
- [8] S. R. White and I. Affleck, “Spectral functions with the density matrix renormalization group,” 2008. cited on p. [2](#), [19](#), [26](#)
- [9] T. Barthel, U. Schollwöck, and S. R. White, “Spectral functions in one-dimensional quantum systems at finite temperature using the density matrix renormalization group,” 2009. cited on p. [2](#), [19](#), [26](#)
- [10] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” 2018. cited on p. [2](#), [21](#), [22](#)
- [11] J. ichi Suzuki, J. ichi Inoue, and B. K. Chakrabarti, *Quantum Ising Phases and Transitions in Transverse Ising Models*, 2012. cited on p. [3](#)
- [12] A. Polkovnikov, K. Sengupta, A. Silva, and M. Vengalattore, “Colloquium: Nonequilibrium dynamics of closed interacting quantum systems,” 2011. cited on p. [8](#), [9](#)
- [13] E. H. Lieb and D. W. Robinson, “The finite group velocity of quantum spin systems,” 1972. cited on p. [11](#)

- [14] M. B. Hastings and T. Koma, “Spectral gap and exponential decay of correlations,” 2006. cited on p. [11](#)
- [15] S. Bravyi, M. B. Hastings, and F. Verstraete, “Lieb–robinson bounds and the generation of correlations and topological quantum order,” 2006. cited on p. [11](#)
- [16] B. Nachtergaele and R. Sims, “Lieb–robinson bounds and the exponential clustering theorem,” 2006. cited on p. [11](#)
- [17] ———, “Lieb–robinson bounds in quantum many-body physics,” 2010. cited on p. [11](#)
- [18] M. B. Hastings, “An area law for one-dimensional quantum systems,” 2007. cited on p. [14](#)
- [19] D. N. Page, “Average entropy of a subsystem,” 1993. cited on p. [14](#)
- [20] P. Calabrese and J. Cardy, “Entanglement entropy and quantum field theory,” 2004. cited on p. [14](#)
- [21] U. Schollwöck, “The density-matrix renormalization group in the age of matrix product states,” 2011. cited on p. [15](#)
- [22] M. Suzuki, “Generalized trotter’s formula and systematic approximants of exponential operators and inner derivations,” 1976. cited on p. [17](#)
- [23] H. F. Trotter, “On the product of semi-groups of operators,” 1959. cited on p. [17](#)
- [24] G. Vidal, “Efficient simulation of one-dimensional quantum many-body systems,” 2004. cited on p. [19](#)
- [25] S. R. White and A. E. Feiguin, “Real-time evolution using the density matrix renormalization group,” 2004. cited on p. [19](#)
- [26] J. Hauschild, J. Unfried, S. Anand, B. Andrews, M. Bintz, U. Borla, S. Divic, M. Drescher, J. Geiger, M. Hefel, K. Hémerly, W. Kadow, J. Kemp, N. Kirchner, V. S. Liu, G. Möller, D. Parker, M. Rader, A. Romen, S. Scalet, L. Schoonderwoerd, M. Schulz, T. Soejima, P. Thoma, Y. Wu, P. Zechmann, L. Zweng, R. S. K. Mong, M. P. Zaletel, and F. Pollmann, “Tensor network Python (TeNPy) version 1,” 2024. cited on p. [23](#)